

Notas sobre Métodos Numéricos con R

Carlos E. Martínez-Rodríguez
Universidad Autónoma de la Ciudad de México
Academia de Matemáticas
`carlos.martinez@uacm.edu.mx`

Índice

1. Raíces de Funciones	1
1.1. Método de Bisección	1
1.1.1. Ejemplos	4
1.1.2. Implementaciones numéricas	9
1.2. Método de la Secante	11
1.3. Método de Newton-Raphson	17
1.4. Método del punto fijo	20
1.5. Ejercicios	24
1.5.1. Método de Bisección	24
1.5.2. Método de la Secante	25
1.5.3. Método de Newton	26
1.5.4. Método del Punto Fijo	27
1.5.5. Para impresión	29
1.5.6. Punto fijo	29
1.5.7. Newton-Raphson	30
1.5.8. Bisección	31
1.5.9. Secante	32
1.6. Tarea para el portafolio	35

1. Raíces de Funciones

1.1. Método de Bisección

Dada una función continua $f : [a, b] \rightarrow \mathbb{R}$ con $f(a)f(b) < 0$, por el **Teorema de Bolzano** existe al menos una raíz en (a, b) . El **método de bisección** explota este cambio de signo: divide el intervalo a la mitad, selecciona la submitad donde persiste el cambio de signo, y repite. Es un método *robusto* (garantiza convergencia si se mantienen las hipótesis) y con *tasa lineal*.

Algoritmo 1. Para encontrar la raíz de una función utilizando el método de la bisección es

1. Verificar continuidad de f en $[a, b]$ (al menos de forma razonable) y que $f(a)f(b) < 0$.
2. Repetir para $n = 1, 2, \dots$:

- a) $c = \frac{a+b}{2}$, evaluar $f(c)$.
- b) Si $f(c) = 0$ (o $|f(c)|$ pequeño), **parar**: c es la raíz aproximada.
- c) Si $f(a)f(c) < 0$, poner $b \leftarrow c$; en caso contrario, $a \leftarrow c$.
- d) Criterio de paro: ancho $(b-a)/2 < \varepsilon$ o $|f(c)| < \varepsilon$, o alcanzar $n_{\text{máx}}$.

Cota del error: Si (a_0, b_0) es el intervalo inicial, tras n iteraciones el punto medio c_n satisface

$$|c_n - \alpha| \leq \frac{b_0 - a_0}{2^n}, \quad (1)$$

donde α es alguna raíz en (a_0, b_0) . Para asegurar $(b_n - a_n)/2 < \varepsilon$, basta con

$$n \geq \left\lceil \log_2 \left(\frac{b_0 - a_0}{\varepsilon} \right) \right\rceil. \quad (2)$$

Resultado 1. Si $f(x)$ es continua en $[a, b]$ y $f(a)f(b) < 0$, por el **Teorema de Bolzano** existe al menos una raíz en (a, b) . El método de bisección consiste en:

$$c_k = \frac{a_k + b_k}{2}, \quad \text{y se actualiza el intervalo de búsqueda} \quad \begin{cases} b_{k+1} = c_k & \text{si } f(a_k)f(c_k) < 0, \\ a_{k+1} = c_k & \text{si } f(a_k)f(c_k) > 0. \end{cases} \quad (3)$$

La cota del error después de n iteraciones es

$$|c_n - \alpha| \leq \frac{b_0 - a_0}{2^n}. \quad (4)$$

Ejemplo 1. # =====

MÉTODO DE BISECCIÓN PARA UNA CÚBICA GENERAL

=====

---- Coeficientes del polinomio cúbico ----

A <- 1 # coeficiente de x³

B <- -6 # coeficiente de x²

C <- 11 # coeficiente de x

D <- -6 # término independiente

---- Definición de la función cúbica ----

f <- function(x) {

 A*x³ + B*x² + C*x + D

}

---- Intervalo inicial ----

a1 <- 0

b1 <- 2.5 # asegúrate que f(a1)*f(b1) < 0

==== Iteración 1 ====

c1 <- (a1 + b1)/2

fa1 <- f(a1)

fb1 <- f(b1)

fc1 <- f(c1)

```

s1a <- sign(fa1 * fc1)
s1b <- sign(fb1 * fc1)
e1 <- (b1 - a1)/2
a2 <- if (fa1 * fc1 < 0) a1 else c1
b2 <- if (fa1 * fc1 < 0) c1 else b1

# ==== Iteración 2 ====
c2 <- (a2 + b2)/2
fa2 <- f(a2)
fb2 <- f(b2)
fc2 <- f(c2)
s2a <- sign(fa2 * fc2)
s2b <- sign(fb2 * fc2)
e2 <- (b2 - a2)/2
a3 <- if (fa2 * fc2 < 0) a2 else c2
b3 <- if (fa2 * fc2 < 0) c2 else b2

# ==== Iteración 3 ====
c3 <- (a3 + b3)/2
fa3 <- f(a3)
fb3 <- f(b3)
fc3 <- f(c3)
s3a <- sign(fa3 * fc3)
s3b <- sign(fb3 * fc3)
e3 <- (b3 - a3)/2
a4 <- if (fa3 * fc3 < 0) a3 else c3
b4 <- if (fa3 * fc3 < 0) c3 else b3

# ==== Iteración 4 ====
c4 <- (a4 + b4)/2
fa4 <- f(a4)
fb4 <- f(b4)
fc4 <- f(c4)
s4a <- sign(fa4 * fc4)
s4b <- sign(fb4 * fc4)
e4 <- (b4 - a4)/2
a5 <- if (fa4 * fc4 < 0) a4 else c4
b5 <- if (fa4 * fc4 < 0) c4 else b4

# ==== Iteración 5 ====
c5 <- (a5 + b5)/2
fa5 <- f(a5)
fb5 <- f(b5)
fc5 <- f(c5)
s5a <- sign(fa5 * fc5)
s5b <- sign(fb5 * fc5)
e5 <- (b5 - a5)/2
a6 <- if (fa5 * fc5 < 0) a5 else c5

```

```

b6 <- if (fa5 * fc5 < 0) c5 else b5

# ==== Iteración 6 ====
c6 <- (a6 + b6)/2
fa6 <- f(a6)
fb6 <- f(b6)
fc6 <- f(c6)
s6a <- sign(fa6 * fc6)
s6b <- sign(fb6 * fc6)
e6 <- (b6 - a6)/2

# ---- Aproximación final ----
raiz_aprox <- c6
error_cota <- e6

cat("Raíz aproximada (6 iteraciones):", raiz_aprox, "\n")
cat("Cota máxima del error:", error_cota, "\n\n")

# ---- Tabla resumen ----
iter <- 1:6
Acol <- c(a1,a2,a3,a4,a5,a6)
Bcol <- c(b1,b2,b3,b4,b5,b6)
Ccol <- c(c1,c2,c3,c4,c5,c6)
FA <- c(fa1,fa2,fa3,fa4,fa5,fa6)
FB <- c(fb1,fb2,fb3,fb4,fb5,fb6)
FC <- c(fc1,fc2,fc3,fc4,fc5,fc6)
signFAFC <- c(s1a,s2a,s3a,s4a,s5a,s6a)
signFBFC <- c(s1b,s2b,s3b,s4b,s5b,s6b)
ERR <- c(e1,e2,e3,e4,e5,e6)

TAB <- data.frame(iter, a=Acol, b=Bcol, c=Ccol,
                  fa=FA, fb=FB, fc=FC,
                  "sign(fa*fc)"=signFAFC,
                  "sign(fb*fc)"=signFBFC,
                  error=ERR)

print(round(TAB, 6))

```

1.1.1. Ejemplos

Ejemplo 2. Consideremos la función $f(x) = x^2 - 5x + 6$. para $a = 1, b = -5, c = 6$.

- Verificar cambio de signo. Evaluamos: $f(1) = 1 - 5 + 6 = 2 > 0$, $f(2.5) = 6.25 - 12.5 + 6 = -0.25 < 0$. Existe cambio de signo entre $x = 1$ y $x = 2.5$, por lo tanto, hay al menos una raíz en $[1, 2.5]$.
- El punto medio inicial es: $c_1 = \frac{1+2.5}{2} = 1.75$, $f(1.75) = 3.0625 - 8.75 + 6 = 0.3125 > 0$. Como $f(1.75) > 0$ y $f(2.5) < 0$, el nuevo intervalo es $[1.75, 2.5]$.

- Segunda iteración: $c_2 = \frac{1.75+2.5}{2} = 2.125$, $f(2.125) = 4,5156 - 10,625 + 6 = -0.1094 < 0$. Nuevo intervalo: $[1.75, 2.125]$.
- Tercera iteración: $c_3 = \frac{1.75+2.125}{2} = 1.9375$, $f(1.9375) = 3,754 - 9,6875 + 6 = 0.0665 > 0$. Nuevo intervalo: $[1.9375, 2.125]$.
- Cuarta iteración: $c_4 = \frac{1.9375+2.125}{2} = 2.03125$, $f(2.03125) = 4,126 - 10,156 + 6 = -0.030 < 0$. Nuevo intervalo: $[1.9375, 2.03125]$.
- Después de unas pocas iteraciones, la raíz se aproxima a $x \approx 2.0$.

Iteración	a_n	b_n	c_n	$f(c_n)$
1	1.000	2.500	1.750	+0.3125
2	1.750	2.500	2.125	-0.1094
3	1.750	2.125	1.9375	+0.0665
4	1.9375	2.125	2.0313	-0.030
5	1.9375	2.0313	1.9844	+0.017

Ejemplo 3. Sea $f(x) = x^2 - 2$ con $a = 1$, $b = -2$. Entonces $f(1) = -1 < 0$, $f(2) = 2 > 0 \Rightarrow$ hay raíz en $[1, 2]$.

Iter.	a_n	b_n	c_n	$f(c_n)$
1	1.000000	2.000000	1.500000	+0.250000
2	1.000000	1.500000	1.250000	-0.437500
3	1.250000	1.500000	1.375000	-0.109375
4	1.375000	1.500000	1.437500	+0.066406
5	1.375000	1.437500	1.406250	-0.022461
6	1.406250	1.437500	1.421875	+0.021729
7	1.406250	1.421875	1.414063	-0.000427
8	1.414063	1.421875	1.417969	+0.010635

Ejemplo 4. Sea la función $f(x) = x^3 - 6x^2 + 11x - 6$, con $a = 1$, $b = -6$, $c = 11$, $d = -6$, en el intervalo $[1.5, 2.5]$.

- Verificar cambio de signo. $f(1.5) = 1.5^3 - 6(1.5)^2 + 11(1.5) - 6 = 3.375 - 13.5 + 16.5 - 6 = 0.375 > 0$, $f(2.5) = 15.625 - 37.5 + 27.5 - 6 = -0.375 < 0$. Existe cambio de signo, por lo que hay al menos una raíz en el intervalo $[1.5, 2.5]$.
- Aplicar el método de bisección. $c_1 = \frac{1.5+2.5}{2} = 2.0$, $f(2.0) = 8 - 24 + 22 - 6 = 0$. Se obtiene exactamente la raíz en la primera iteración.
- Para ilustrar mejor, tomemos el intervalo más amplio $[1, 3]$: $f(1) = 0$, $f(3) = 0$. Aquí no hay cambio de signo estricto (ya que ambos extremos son raíces), por tanto el método se aplica en un subintervalo, por ejemplo $[1.2, 2.8]$.
- Iteraciones del método:

Iteración	a_n	b_n	c_n	$f(c_n)$
1	1.200	2.800	2.000	0.000

- *Nuevas evaluaciones:* $f(1.5) = 3.375 - 13.5 + 16.5 - 5.5 = 0.875 > 0$, $f(2.5) = 15.625 - 37.5 + 27.5 - 5.5 = 0.125 > 0$, $f(1.0) = 1 - 6 + 11 - 5.5 = 0.5 > 0$, $f(3.0) = 27 - 54 + 33 - 5.5 = 0.5 > 0$.
- *Busquemos ahora en el intervalo $[1.5, 2.5]$ ajustando un poco los valores hasta encontrar cambio de signo:* $f(1.7) \approx 0.28$, $f(1.9) \approx 0.03$, $f(2.0) \approx -0.5$. Entonces hay cambio de signo en $[1.8, 2.0]$.

Iteración	a_n	b_n	c_n	$f(c_n)$
1	1.800	2.000	1.900	+0.030
2	1.900	2.000	1.950	-0.210
3	1.800	1.950	1.875	+0.130
4	1.875	1.950	1.9125	-0.040
5	1.875	1.9125	1.8937	+0.040
6	1.8937	1.9125	1.9031	-0.005

La raíz aproximada se encuentra en $x \approx 1.903$.

Ejemplo 5. Consideremos la cúbica $f(x) = x^3 - 6x^2 + 11x - 5.5$, con intervalo inicial $[a_0, b_0] = [0.8, 0.9]$ donde hay cambio de signo. Las primeras 8 iteraciones (valores redondeados) son:

Iter.	a_n	b_n	c_n	$f(c_n)$
1	0.8000000	0.9000000	0.8500000	$+1.29125 \times 10^{-1}$
2	0.8000000	0.8500000	0.8250000	$+5.27656 \times 10^{-2}$
3	0.8000000	0.8250000	0.8125000	$+1.29395 \times 10^{-2}$
4	0.8000000	0.8125000	0.8062500	-7.39038×10^{-3}
5	0.8062500	0.8125000	0.8093750	$+2.80942 \times 10^{-3}$
6	0.8062500	0.8093750	0.8078125	-2.28175×10^{-3}
7	0.8078125	0.8093750	0.8085938	$+2.66016 \times 10^{-4}$
8	0.8078125	0.8085938	0.8082031	-1.00732×10^{-3}

Después de 8 iteraciones, la aproximación de la raíz es $c_8 \approx 0.8082031$ y la cota de error por ancho de intervalo es $\frac{b_8 - a_8}{2} = \text{error_cota} = \frac{0.8085938 - 0.8078125}{2} \approx 3.906 \times 10^{-4}$.

Ejemplo 6. Sea $f(x) = x^2 - 2x$ con $a = 1$ y $b = -3$ pues $ax^2 + bx + x = x^2 - 3x + x = x^2 - 2x$. Usamos el intervalo $[1.2, 3.0]$: $f(1.2) = 1.44 - 2.4 = -0.96 < 0$, $f(3) = 9 - 6 = 3 > 0$.

Iter.	a_n	b_n	c_n	$f(c_n)$
1	1.200000	3.000000	2.100000	+0.210000
2	1.200000	2.100000	1.650000	-0.577500
3	1.650000	2.100000	1.875000	-0.234375
4	1.875000	2.100000	1.987500	-0.024844
5	1.987500	2.100000	2.043750	+0.089414
6	1.987500	2.043750	2.015625	+0.031494
7	1.987500	2.015625	2.001563	+0.003127
8	1.987500	2.001563	1.994531	-0.010908

Ejemplo 7. Sea la función $f(x) = x^2 - 2x$ en el intervalo elegido: $[1.2, 3.0]$

- $f(1.2) = 1.44 - 2.4 = -0.96$, $f(3) = 9 - 6 = 3$.
- Iteración 1: $c_1 = \frac{1.2+3}{2} = 2.1$, $f(2.1) = 4.41 - 4.2 = 0.21 > 0$ entonces $[1.2, 2.1]$.

- Iteración 2: $c_2 = \frac{1.2+2.1}{2} = 1.65$, $f(1.65) = 2.7225 - 3.3 = -0.5775 < 0$ entonces $[1.65, 2.1]$.
- Iteración 3: $c_3 = 1.875$, $f(1.875) = 3.516 - 3.75 = -0.234 < 0$ entonces $[1.875, 2.1]$. Y así sucesivamente hasta $c_8 \approx 1.9945$.

n	a_n	b_n	c_n	$f(c_n)$
1	1.200	3.000	2.100	+0.210
2	1.200	2.100	1.650	-0.577
3	1.650	2.100	1.875	-0.234
4	1.875	2.100	1.987	-0.024
5	1.987	2.100	2.043	+0.089
6	1.987	2.043	2.015	+0.031
7	1.987	2.015	2.001	+0.003
8	1.987	2.001	1.994	-0.011

Ejemplo 8. $f(x) = x^3 - x - 2$ en $[1, 2]$. Se verifica $f(1) = -2 < 0$ y $f(2) = 4 > 0$.

Primeras iteraciones (hasta alcanzar 10^{-6} típicamente se requieren ~ 20 pasos desde $[1, 2]$):

Cuadro 1: Método de bisección para $f(x) = x^3 - x - 2$, intervalo inicial $[-3, 2]$.

Iter.	a	b	$c = \frac{a+b}{2}$	$f(a)$	$f(b)$	$f(c)$	$\text{sign}(f(a)f(c))$	$\text{sign}(f(b)f(c))$
1	-3.000000000	2.000000000	-0.500000000	-26.000000000	4.000000000	-1.625000000	+	-
2	-0.500000000	2.000000000	0.750000000	-1.625000000	4.000000000	-2.328125000	+	-
3	0.750000000	2.000000000	1.375000000	-2.328125000	4.000000000	-0.775390625	+	-
4	1.375000000	2.000000000	1.687500000	-0.775390625	4.000000000	1.117919922	-	+
5	1.375000000	1.687500000	1.531250000	-0.775390625	1.117919922	0.059112549	-	+
6	1.375000000	1.531250000	1.453125000	-0.775390625	0.059112549	-0.392456055	+	-
7	1.453125000	1.531250000	1.492187500	-0.392456055	0.059112549	-0.172897339	+	-
8	1.492187500	1.531250000	1.511718750	-0.172897339	0.059112549	-0.058979273	+	-
9	1.511718750	1.531250000	1.521484375	-0.058979273	0.059112549	0.000622176	-	+
10	1.511718750	1.521484375	1.516601562	-0.058979273	0.000622176	-0.029292628	+	-
11	1.516601562	1.521484375	1.519042969	-0.029292628	0.000622176	-0.013864168	+	-
12	1.519042969	1.521484375	1.520263672	-0.013864168	0.000622176	-0.006627792	+	-

Con tolerancia $\varepsilon = 10^{-6}$, el resultado se estabiliza alrededor de $\alpha \approx 1,52138$.

Ejemplo 9. $f(x) = \cos x - x$ en $[0, 1]$. Se verifica $f(0) = 1 > 0$ y $f(1) \approx -0,4597 < 0$.

Primeras iteraciones:

Cuadro 2: Método de bisección para $f(x) = \cos x - x$ en $[0, 1]$.

Iter.	a	b	$c = \frac{a+b}{2}$	$f(a)$	$f(b)$	$f(c)$	$\text{sign}(f(a)f(c))$	$\text{sign}(f(b)f(c))$
1	0.000000000	1.000000000	0.500000000	1.000000000	-0.459697694	0.377582562	-	+
2	0.500000000	1.000000000	0.750000000	0.377582562	-0.459697694	-0.018311132	+	-
3	0.500000000	0.750000000	0.625000000	0.377582562	-0.018311132	0.185963120	-	+
4	0.625000000	0.750000000	0.687500000	0.185963120	-0.018311132	0.085334946	-	+
5	0.687500000	0.750000000	0.718750000	0.085334946	-0.018311132	0.033879372	-	+
6	0.718750000	0.750000000	0.734375000	0.033879372	-0.018311132	0.007874725	-	+
7	0.734375000	0.750000000	0.742187500	0.007874725	-0.018311132	-0.005195711	+	-
8	0.734375000	0.742187500	0.738281250	0.007874725	-0.005195711	0.001345150	-	+
9	0.738281250	0.742187500	0.740234375	0.001345150	-0.005195711	-0.001923872	+	-
10	0.738281250	0.740234375	0.739257812	0.001345150	-0.001923872	-0.000289009	+	-
11	0.738281250	0.739257812	0.738769531	0.001345150	-0.000289009	0.000528158	-	+
12	0.738769531	0.739257812	0.739013672	0.000528158	-0.000289009	0.000119610	-	+

Con $\varepsilon = 10^{-6}$ se obtiene aproximadamente $\alpha \approx 0,739085$.

Ejemplo 10. Aplica el método de la bisección para las siguientes funciones:

1. Encuentra una raíz de $x^3 - 6x + 2$ en $[0, 2]$.

Cuadro 3: Método de la Bisección para $f(x) = x^3 - 6x + 2$ en $[0, 2]$ (12 iteraciones).

Iter.	a	b	$c = \frac{a+b}{2}$	$f(a)$	$f(b)$	$f(c)$	$\text{sign}(f(a)f(c))$	$\text{sign}(f(b)f(c))$
1	0.000000000	2.000000000	1.000000000	2.000000000	-2.000000000	-3.000000000	-	+
2	0.000000000	1.000000000	0.500000000	2.000000000	-3.000000000	-0.875000000	-	+
3	0.000000000	0.500000000	0.250000000	2.000000000	-0.875000000	0.515625000	+	-
4	0.250000000	0.500000000	0.375000000	0.515625000	-0.875000000	-0.197265625	-	+
5	0.250000000	0.375000000	0.312500000	0.515625000	-0.197265625	0.155517578	+	-
6	0.312500000	0.375000000	0.343750000	0.155517578	-0.197265625	-0.021881104	-	+
7	0.312500000	0.343750000	0.328125000	0.155517578	-0.021881104	0.066577911	+	-
8	0.328125000	0.343750000	0.335937500	0.066577911	-0.021881104	0.022286892	+	-
9	0.335937500	0.343750000	0.339843750	0.022286892	-0.021881104	0.000187337	+	-
10	0.339843750	0.343750000	0.341796875	0.000187337	-0.021881104	-0.010850795	-	+
11	0.339843750	0.341796875	0.340820312	0.000187337	-0.010850795	-0.005332704	-	+
12	0.339843750	0.340820312	0.340332031	0.000187337	-0.005332704	-0.002572927	-	+

2. Estudia $f(x) = e^{-x} - x$ en $[0, 1]$.

Cuadro 4: Método de Bisección para $f(x) = e^{-x} - x$ en $[0, 1]$ (12 iteraciones).

Iter.	a	b	$c = \frac{a+b}{2}$	$f(a)$	$f(b)$	$f(c)$	$\text{sign}(f(a)f(c))$	$\text{sign}(f(b)f(c))$
1	0.000000000	1.000000000	0.500000000	1.000000000	-0.632120559	0.106530660	+	-
2	0.500000000	1.000000000	0.750000000	0.106530660	-0.632120559	-0.277633447	-	+
3	0.500000000	0.750000000	0.625000000	0.106530660	-0.277633447	-0.089738571	-	+
4	0.500000000	0.625000000	0.562500000	0.106530660	-0.089738571	0.007282825	+	-
5	0.562500000	0.625000000	0.593750000	0.007282825	-0.089738571	-0.041497550	-	+
6	0.562500000	0.593750000	0.578125000	0.007282825	-0.041497550	-0.017175839	-	+
7	0.562500000	0.578125000	0.570312500	0.007282825	-0.017175839	-0.004963760	-	+
8	0.562500000	0.570312500	0.566406250	0.007282825	-0.004963760	0.001155202	+	-
9	0.566406250	0.570312500	0.568359375	0.001155202	-0.004963760	-0.001905360	-	+
10	0.566406250	0.568359375	0.567382812	0.001155202	-0.001905360	-0.000375349	-	+
11	0.566406250	0.567382812	0.566894531	0.001155202	-0.000375349	0.000389859	+	-
12	0.566894531	0.567382812	0.567138672	0.000389859	-0.000375349	0.000007238	+	-

3. Sea la función cuadrática $f(x) = x^2 - 4$ en $(-1, 4)$.

Cuadro 5: Método de Bisección para $f(x) = x^2 - 4$ en $(-1, 4)$ (12 iteraciones).

Iter.	a	b	$c = \frac{a+b}{2}$	$f(a)$	$f(b)$	$f(c)$	$\text{sign}(f(a)f(c))$	$\text{sign}(f(b)f(c))$
1	-1.000000000	4.000000000	1.500000000	-3.000000000	12.000000000	-1.750000000	+	-
2	1.500000000	4.000000000	2.750000000	-1.750000000	12.000000000	3.562500000	-	+
3	1.500000000	2.750000000	2.125000000	-1.750000000	3.562500000	0.515625000	-	+
4	1.500000000	2.125000000	1.812500000	-1.750000000	0.515625000	-0.714843750	+	-
5	1.812500000	2.125000000	1.968750000	-0.714843750	0.515625000	-0.122558594	+	-
6	1.968750000	2.125000000	2.046875000	-0.122558594	0.515625000	0.189208984	-	+
7	1.968750000	2.046875000	2.007812500	-0.122558594	0.189208984	0.031494141	-	+
8	1.968750000	2.007812500	1.988281250	-0.122558594	0.031494141	-0.063354492	+	-
9	1.988281250	2.007812500	1.998046875	-0.063354492	0.031494141	-0.007873535	+	-
10	1.998046875	2.007812500	2.002929688	-0.007873535	0.031494141	0.011726379	-	+
11	1.998046875	2.002929688	2.000488281	-0.007873535	0.011726379	0.001953602	-	+
12	1.998046875	2.000488281	1.999267578	-0.007873535	0.001953602	-0.002960747	+	-

1.1.2. Implementaciones numéricas

Algoritmo 2. *Pseudocódigo*

```
Inicio
  Definir f(x)
  Leer a, b, tolerancia, iter_max

  Si f(a)*f(b) > 0 Entonces
    Imprimir "No hay cambio de signo en [a,b]"
    Terminar
  FinSi

  iter ← 0
  error ← |b - a|

  Mientras (error > tolerancia) Y (iter < iter_max) Hacer
    c ← (a + b)/2

    Si f(a)*f(c) < 0 Entonces
      b ← c
    Sino
      a ← c
    FinSi

    error ← |b - a|
    iter ← iter + 1
  FinMientras

  Imprimir "Raíz aproximada:", c
  Imprimir "Iteraciones:", iter
Fin
```

Implementación numérica

```
# Método de Bisección
biseccion <- function(f, a, b, tol = 1e-6, iter_max = 100){
  if(f(a)*f(b) > 0){
    cat("Error: no hay cambio de signo en el intervalo [a,b].\n")
    return(NA)
  }

  iter <- 0
  error <- abs(b - a)

  while(error > tol && iter < iter_max){
    c <- (a + b)/2
```

```

    if(f(a)*f(c) < 0){
      b <- c
    } else {
      a <- c
    }

    error <- abs(b - a)
    iter <- iter + 1
  }

  cat("Raíz aproximada:", c, "\n")
  cat("Iteraciones:", iter, "\n")
  return(c)
}

```

```

# Ejemplo de uso:
# f(x) = x^3 - x - 2
f <- function(x) x^3 - x - 2
biseccion(f, a = 1, b = 2)
\left

```

Algoritmo 3. # Definir f(x) (la función matemática a evaluar)

```
f <- function(x) x^3 - 6*x^2 + 11*x - 5.5
```

```
# Intervalo inicial con cambio de signo
```

```
a <- 0.8; b <- 0.9
```

```
fa <- f(a); fb <- f(b)
```

```
if (fa * fb >= 0) stop("El intervalo inicial no tiene cambio de signo.")
```

```
# Estructura para guardar el historial (n, a, b, c, f(c), ancho)
```

```
hist <- data.frame(
  n = integer(), a = numeric(), b = numeric(),
  c = numeric(), fc = numeric(), ancho = numeric(),
  stringsAsFactors = FALSE
)
```

```
# Ejecutar exactamente 8 iteraciones, sin usar ninguna funcion/recursividad
```

```
for (n in 1:8) {
```

```
  c <- (a + b)/2
```

```
  fc <- f(c)
```

```
  ancho <- (b - a)/2
```

```
# Guardar fila
```

```
hist[n,] <- list(n, a, b, c, fc, ancho)
```

```
# Mostrar en consola (opcional)
```

```
cat(sprintf("Iter %2d | a=%.7f b=%.7f c=%.7f f(c)=%.7e ancho=%.7e\n",
            n, a, b, c, fc, ancho))
```

```

# Actualizar el intervalo según el signo
if (fa * fc < 0) {
  b <- c; fb <- fc
} else {
  a <- c; fa <- fc
}
}

# Resultado aproximado tras 8 iteraciones:
c_aprox <- hist$c[8]
error_cota <- hist$ancho[8] # (b - a)/2 después del paso 8
c_aprox; error_cota
hist

```

Algoritmo 4. `f <- function(x) x^2 - 2`

```

a <- 1.0; b <- 2.0
fa <- f(a); fb <- f(b)
if (fa*fb >= 0) stop("Sin cambio de signo en [a,b].")
histA <- data.frame(n=integer(), a=numeric(), b=numeric(),
                    c=numeric(), fc=numeric(), ancho=numeric())
for (n in 1:8) {
  c <- (a + b)/2
  fc <- f(c)
  histA[n,] <- list(n, a, b, c, fc, (b-a)/2)
  cat(sprintf("A-%2d | a=%.6f b=%.6f c=%.6f f(c)=%.6e ancho=%.6e\n",
              n, a, b, c, fc, (b-a)/2))
  if (fa * fc < 0) { b <- c; fb <- fc } else { a <- c; fa <- fc }
}

```

Algoritmo 5. `f <- function(x) x^2 - 2*x`

```

a <- 1.2; b <- 3.0
fa <- f(a); fb <- f(b)
if (fa*fb >= 0) stop("Sin cambio de signo en [a,b].")
histB <- data.frame(n=integer(), a=numeric(), b=numeric(),
                    c=numeric(), fc=numeric(), ancho=numeric())
for (n in 1:8) {
  c <- (a + b)/2
  fc <- f(c)
  histB[n,] <- list(n, a, b, c, fc, (b-a)/2)
  cat(sprintf("B-%2d | a=%.6f b=%.6f c=%.6f f(c)=%.6e ancho=%.6e\n",
              n, a, b, c, fc, (b-a)/2))
  if (fa * fc < 0) { b <- c; fb <- fc } else { a <- c; fa <- fc }
}

```

1.2. Método de la Secante

El método de la secante aproxima la derivada mediante la pendiente de la recta que une dos puntos de la curva, evitando calcular $f'(x)$.

Algoritmo 6. Dado x_{k-1} y x_k , definimos

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

- **Criterios de paro:** $|x_{k+1} - x_k| < \varepsilon$ o $|f(x_{k+1})| < \varepsilon$.
- **Ventajas:** no requiere derivada; suele ser más rápido que bisección.
- **Sugerencia:** elegir dos semillas razonables; evitar $f(x_k) \approx f(x_{k-1})$ (división por número muy pequeño).

Ejemplo 11. Función Cuadrática: $f(x) = x^2 - 4$. Semillas: $x_0 = 1$, $x_1 = 3$. Buscamos la raíz positiva (2). Consideremos la aproximación inicial x_0, x_1 y el nuevo punto x_2 es el punto donde se intersecta la secante con el eje x .

$$f(x) = x^2 - 4, \quad f(1) = -3, \quad f(3) = 5.$$

- Iteración 1.

$$x_2 = 3 - 5 \frac{(3 - 1)}{5 - (-3)} = 1.75, \quad |x_2 - x_1| = 1.25.$$

- Iteración 2. $x_0 = 3$, $x_1 = 1.75$, $f(1.75) = -0.9375$,

$$x_3 = 1.75 - (-0.9375) \frac{(1.75 - 3)}{-0.9375 - 5} = 1.947368421.$$

- Iteración 3. $f(1.947368421) = -0.207756233$,

$$x_4 = 1.947368421 - (-0.207756233) \frac{(1.947368421 - 1.75)}{-0.207756233 - (-0.9375)} = 2.003558719.$$

- Iteración 4. $f(2.003558719) = 0.014247540$,

$$x_5 = 2.003558719 - (0.014247540) \frac{(2.003558719 - 1.947368421)}{0.014247540 - (-0.207756233)} = 1.999952593.$$

- Iteración 5. $f(1.999952593) = -1.89625 \times 10^{-4}$, y $x_6 = 1.999999958$, por lo tanto $x \approx 2.000000000$.

Solución en R:

```
# f(x) = x^2 - 4    (x0=1, x1=3)
x0 <- 1.0; x1 <- 3.0
# Iteración 1
f0 <- x0^2 - 4; f1 <- x1^2 - 4
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 2
f0 <- x0^2 - 4; f1 <- x1^2 - 4
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
```

```

# Iteración 3
f0 <- x0^2 - 4; f1 <- x1^2 - 4
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 4
f0 <- x0^2 - 4; f1 <- x1^2 - 4
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 5
f0 <- x0^2 - 4; f1 <- x1^2 - 4
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
root_aprox <- x2
print(root_aprox)

```

Ejemplo 12. $f(x) = x^3 - x - 2$. Aproximación inicial: $x_0 = 1$, $x_1 = 2$, entonces

$$f(1) = -2, \quad f(2) = 4.$$

■ *Iteración 1.*

$$x_2 = 2 - (4) \frac{(2 - 1)}{4 - (-2)} = 1.333333333.$$

■ *Iteración 2.* $f(1.333333333) = -0.962962963$; $x_3 = 1.462686567$.

■ *Iteración 3.* $f(1.462686567) = -0.333338875$; $x_4 = 1.531169432$.

■ *Iteración 4.* $f(1.531169432) = 0.058626418$; $x_5 = 1.520926421$.

■ *Iteración 5.* $f(1.520926421) = -0.002693300$; $x_6 = 1.521376317$;

Solución en R

```

# f(x) = x^3 - x - 2    (x0=1, x1=2)
x0 <- 1.0; x1 <- 2.0
# Iteración 1
f0 <- x0^3 - x0 - 2; f1 <- x1^3 - x1 - 2
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 2
f0 <- x0^3 - x0 - 2; f1 <- x1^3 - x1 - 2
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 3
f0 <- x0^3 - x0 - 2; f1 <- x1^3 - x1 - 2
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 4
f0 <- x0^3 - x0 - 2; f1 <- x1^3 - x1 - 2
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2

```

```
# Iteración 5
f0 <- x0^3 - x0 - 2; f1 <- x1^3 - x1 - 2
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
root_aprox <- x2
print(root_aprox)
```

Ejemplo 13. $f(x) = \cos x - x$. Aproximación inicial: $x_0 = 0.5$, $x_1 = 1.0$, por tanto $f(0.5) = 0.37758256$ y $f(1.0) = -0.45969769$.

- Iteración 1. $x_2 = 0.725481587$, $f(x_2) \approx 0.022698391$.
- Iteración 2. $x_3 = 0.738398620$, $f(x_3) \approx 0.001148782$.
- Iteración 3. $x_4 = 0.739087211$, $f(x_4) \approx -3.4771 \times 10^{-6}$.
- Iteración 4. $x_5 = 0.739085133$; entonces $x \approx 0.739085133$.

Solución en R

```
# f(x) = cos(x) - x    (x0=0.5, x1=1.0)
x0 <- 0.5; x1 <- 1.0
# Iteración 1
f0 <- cos(x0) - x0; f1 <- cos(x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 2
f0 <- cos(x0) - x0; f1 <- cos(x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 3
f0 <- cos(x0) - x0; f1 <- cos(x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 4
f0 <- cos(x0) - x0; f1 <- cos(x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
root_aprox <- x2
print(root_aprox)
```

Ejemplo 14. $f(x) = e^{-x} - x$. Aproximación inicial: $x_0 = 0$, $x_1 = 1$, por tanto $f(0) = 1$, $f(1) = e^{-1} - 1 \approx -0.632120559$.

- Iteración 1.* $x_2 = 0.612699837$, $f(x_2) \approx -0.070813948$.
- Iteración 2.* $x_3 = 0.563838389$, $f(x_3) \approx 0.005182355$.
- Iteración 3.* $x_4 = 0.567170358$, $f(x_4) \approx -4.2419 \times 10^{-5}$.
- Iteración 4.* $x_5 = 0.567143307$, $f(x_5) \approx -2.5380 \times 10^{-8}$.
- Iteración 5.* $x_6 = 0.567143290$, $x \approx 0.567143290$.

Solución con R

```

# f(x) = exp(-x) - x    (x0=0, x1=1)
x0 <- 0.0; x1 <- 1.0
# Iteración 1
f0 <- exp(-x0) - x0; f1 <- exp(-x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 2
f0 <- exp(-x0) - x0; f1 <- exp(-x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 3
f0 <- exp(-x0) - x0; f1 <- exp(-x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 4
f0 <- exp(-x0) - x0; f1 <- exp(-x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
x0 <- x1; x1 <- x2
# Iteración 5
f0 <- exp(-x0) - x0; f1 <- exp(-x1) - x1
x2 <- x1 - f1*(x1 - x0)/(f1 - f0)
root_aprox <- x2
print(root_aprox)

```

Algoritmo 7. *Pseudocódigo*

Algoritmo: Método de la Secante

Entrada:

f(x)	→ función
x0, x1	→ valores iniciales
tol	→ tolerancia (por ejemplo, 1e-6)
iter_max	→ número máximo de iteraciones

Proceso:

```

iter ← 0
error ← 1

```

```

Mientras (error > tol) Y (iter < iter_max) Hacer
    fx0 ← f(x0)
    fx1 ← f(x1)

```

```

    Si |fx1 - fx0| < 1e-12 Entonces
        Imprimir "Error: división por cero"
        Salir
    FinSi

```

```

    x2 ← x1 - fx1 * (x1 - x0) / (fx1 - fx0)
    error ← |x2 - x1|

```

```

    x0 ← x1
    x1 ← x2
    iter ← iter + 1
FinMientras

```

Salida:

```

    Imprimir "Raíz aproximada:", x2
    Imprimir "Iteraciones:", iter
FinAlgoritmo

```

Implementación numérica

Método de la Secante

```

secante <- function(f, x0, x1, tol = 1e-6, iter_max = 100){
  iter <- 0
  error <- 1

  while(error > tol && iter < iter_max){
    fx0 <- f(x0)
    fx1 <- f(x1)

    # Evitar división por cero
    if(abs(fx1 - fx0) < 1e-12){
      cat("Error: División por cero (f(x1) - f(x0) approx 0)\n")
      return(NA)
    }

    x2 <- x1 - fx1 * (x1 - x0) / (fx1 - fx0)
    error <- abs(x2 - x1)

    # Actualizar valores
    x0 <- x1
    x1 <- x2
    iter <- iter + 1
  }

  cat("Raíz aproximada:", x2, "\n")
  cat("Iteraciones:", iter, "\n")
  return(x2)
}

```

Ejemplo de uso:

```

# f(x) = x^3 - 2x^2 + 3x - 5
f <- function(x) x^3 - 2*x^2 + 3*x - 5
secante(f, x0 = 1, x1 = 2)

```


1.3. Método de Newton-Raphson

Se busca una raíz de $f(x) = 0$ usando la recta *tangente* en x_k . El siguiente punto está dado por

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \quad (5)$$

El criterio de paro es $|x_{k+1} - x_k| < \varepsilon$ o $|f(x_{k+1})| < \varepsilon$.

Algoritmo 8. Entrada: $f(x)$, $f'(x)$, x_0 , tolerancia ε , $N_{\max}$

$x = x_0$

Para $k = 1..N_{\max}$:

$fx = f(x)$; $dfx = f'(x)$

 si $dfx = 0$: detener (tangente horizontal)

$x1 = x - fx/dfx$

 si $|x1 - x| < \varepsilon$ o $|f(x1)| < \varepsilon$: retornar $x1$

$x = x1$

Fin

Ejemplo 15. $f(x) = x^2 - 4$, $f'(x) = 2x$. Aproximación inicial $x_0 = 3$

$$x_1 = 3 - \frac{9 - 4}{2 \cdot 3} = 3 - \frac{5}{6} = 2.166666667.$$

$$x_2 = 2.166666667 - \frac{(2.166666667)^2 - 4}{2 \cdot 2.166666667} = 2.166666667 - \frac{0.694444444}{4.333333334} = 2.006410256.$$

$$x_3 = 2.006410256 - \frac{(2.006410256)^2 - 4}{2 \cdot 2.006410256} \approx 2.000006410.$$

$$x_4 \approx 2.000000000.$$

Raíz aproximada $x \approx 2$.

Solución con R:

```
# f(x) = x^2 - 4, f'(x) = 2*x, x0 = 3; x <- 3.0
# k=1
fx <- x^2 - 4; dfx <- 2*x
x1 <- x - fx/dfx; err <- abs(x1 - x); x <- x1
# k=2
fx <- x^2 - 4; dfx <- 2*x
x1 <- x - fx/dfx; err <- abs(x1 - x); x <- x1
# k=3
fx <- x^2 - 4; dfx <- 2*x
x1 <- x - fx/dfx; err <- abs(x1 - x); x <- x1
root_aprox <- x
print(root_aprox)
```

Ejemplo 16. $f(x) = x^3 - x - 2$, $f'(x) = 3x^2 - 1$. Valor inicial $x_0 = 1.5$

$$\begin{aligned}
f(1.5) &= -0.125; f'(1.5) = 5.75, \\
x_1 &= 1.5 - \frac{-0.125}{5.75} = 1.521739130. \\
f(1.521739130) &\approx 0.002137; f'(1.521739130) \approx 5.947070, \\
x_2 &= 1.521739130 - \frac{0.002137}{5.947070} \approx 1.521380215. \\
f(1.521380215) &\approx 6.03 \times 10^{-7}, f'(1.521380215) \approx 5.943790, \\
x_3 &\approx 1.521380005, \\
x_4 &\approx 1.521379706.
\end{aligned}$$

por tanto $x \approx 1.521379706$. Solución con R

```
# f(x) = x^3 - x - 2, f'(x) = 3*x^2 - 1, x0 = 1.5
x <- 1.5
```

```
# k=1
fx <- x^3 - x - 2; dfx <- 3*x^2 - 1
x1 <- x - fx/dfx; err <- abs(x1 - x); x <- x1
```

```
# k=2
fx <- x^3 - x - 2; dfx <- 3*x^2 - 1
x1 <- x - fx/dfx; err <- abs(x1 - x); x <- x1
```

```
# k=3
fx <- x^3 - x - 2; dfx <- 3*x^2 - 1
x1 <- x - fx/dfx; err <- abs(x1 - x); x <- x1
```

```
root_aprox <- x
print(root_aprox)
```

Ejemplo 17. $f(x) = \cos x - x$, $f'(x) = -\sin x - 1$. Aproximación inicial $x_0 = 0.5$

$$\begin{aligned}
f(0.5) &= 0.37758256, f'(0.5) = -\sin(0.5) - 1 \approx -1.47942554, \\
x_1 &= 0.5 - \frac{0.37758256}{-1.47942554} = 0.755222. \\
f(0.755222) &\approx -0.027103, f'(0.755222) \approx -1.685451, \\
x_2 &= 0.755222 - \frac{-0.027103}{-1.685451} \approx 0.739142. \\
f(0.739142) &\approx -9.46 \times 10^{-5}, f'(0.739142) \approx -1.673654, \\
x_3 &\approx 0.739085; \\
x_4 &\approx 0.739085133.
\end{aligned}$$

por lo tanto $x \approx 0.739085133$.

Ejemplo 18. $f(x) = e^{-x} - x$, $f'(x) = -e^{-x} - 1$. Aproximación inicial $x_0 = 0$

$$\begin{aligned} f(0) &= 1, \quad f'(0) = -2 \Rightarrow x_1 = 0 - \frac{1}{-2} = 0.5. \\ f(0.5) &= e^{-0.5} - 0.5 \approx 0.10653066, \quad f'(0.5) = -e^{-0.5} - 1 \approx -1.60653066, \\ x_2 &= 0.5 - \frac{0.10653066}{-1.60653066} \approx 0.566311. \\ f(0.566311) &\approx 0.001157, \quad f'(0.566311) \approx -1.567468, \\ x_3 &\approx 0.567049, \quad x_4 \approx 0.56714329. \end{aligned}$$

por lo tanto la raíz se encuentra en $x \approx 0.56714329$.

Algoritmo 9. Pseudocódigo

Inicio

```
Definir f(x) y f'(x)
Leer x0, tolerancia, iter_max
iter ← 0
error ← 1
```

```
Mientras (error > tolerancia) Y (iter < iter_max) Hacer
    x1 ← x0 - f(x0)/f'(x0)
    error ← |x1 - x0|
    x0 ← x1
    iter ← iter + 1
```

FinMientras

```
Imprimir "Raíz aproximada:", x1
Imprimir "Iteraciones:", iter
```

Fin

Implementación numérica

Método de Newton-Raphson

```
newton <- function(f, df, x0, tol = 1e-6, iter_max = 100){
  iter <- 0
  error <- 1

  while(error > tol && iter < iter_max){
    x1 <- x0 - f(x0)/df(x0)
    error <- abs(x1 - x0)
    x0 <- x1
    iter <- iter + 1
  }

  cat("Raíz aproximada:", x1, "\n")
  cat("Iteraciones:", iter, "\n")
  return(x1)
}
```

```
# Ejemplo de uso:
# f(x) = x^2 - 2
f <- function(x) x^2 - 2
df <- function(x) 2*x
newton(f, df, x0 = 1)
```

1.4. Método del punto fijo

Dada una ecuación $f(x) = 0$, elegimos una función g tal que

$$f(x) = 0 \quad \text{sí y sólo sí } x = g(x).$$

El método de **punto fijo** construye la sucesión

$$x_{k+1} = g(x_k), \quad k = 0, 1, 2, \dots$$

Teorema 1. Si g es continua en un intervalo I que contiene a la raíz r , $g(I) \subset I$ y $\max_{x \in I} |g'(x)| = L < 1$, entonces existe un único punto fijo $r \in I$ y para $x_0 \in I$ se cumple $x_k \rightarrow r$ con convergencia al menos lineal (factor $\leq L$).

Algoritmo 10. Entrada: $g(x)$, x_0 , tolerancia eps , $N_{\max}$

$x = x_0$

Para $k = 1..N_{\max}$:

$x_1 = g(x)$

 si $|x_1 - x| < \text{eps}$ o $|f(x_1)| < \text{eps}$: retornar x_1

$x = x_1$

Fin

Ejemplo 19. $f(x) = x^2 - 4 = 0$

$$g(x) = \frac{1}{2} \left(x + \frac{4}{x} \right) \quad (\text{iteración de Herón para } \sqrt{4}).$$

Entonces $x_{k+1} = g(x_k)$ y el punto fijo es $r = 2$. Tomamos $x_0 = 3$.

$$\begin{aligned} x_1 &= \frac{1}{2} \left(3 + \frac{4}{3} \right) = 2.166666667. \\ x_2 &= \frac{1}{2} \left(2.166666667 + \frac{4}{2.166666667} \right) = 2.006410256. \\ x_3 &= \frac{1}{2} \left(2.006410256 + \frac{4}{2.006410256} \right) = 2.000003626. \\ x_4 &= \frac{1}{2} \left(2.000003626 + \frac{4}{2.000003626} \right) = 2.000000000. \\ x_5 &\approx 2.000000000. \end{aligned}$$

por lo tanto $r \approx 2$. Aquí $|g'(r)| = \frac{1}{2} \left(1 - \frac{4}{r^2} \right) = 0$, por eso converge muy rápido. Solución con R

```
# g(x) = 0.5*(x + 4/x)    (x0 = 3)
x <- 3.0
```

```
# Iteración 1
x1 <- 0.5*(x + 4/x); x <- x1
```

```
# Iteración 2
x1 <- 0.5*(x + 4/x); x <- x1
```

```
# Iteración 3
x1 <- 0.5*(x + 4/x); x <- x1
```

```
# Iteración 4
x1 <- 0.5*(x + 4/x); x <- x1
```

```
# Iteración 5
x1 <- 0.5*(x + 4/x); x <- x1
```

```
root_aprox <- x
print(root_aprox)
```

Ejemplo 20. $f(x) = x^3 - x - 2 = 0$. Reescribimos $x = \sqrt[3]{x+2}$

$$g(x) = (x+2)^{1/3}.$$

Cerca de la raíz $r \approx 1.52138$, $|g'(r)| = \frac{1}{3}(r+2)^{-2/3} < 1$. Tomamos $x_0 = 1$.

$$\begin{aligned} x_1 &= (1+2)^{1/3} = 1.44224957. \\ x_2 &= (1.44224957+2)^{1/3} = 1.51571657. \\ x_3 &= (1.51571657+2)^{1/3} = 1.52137900. \\ x_4 &= (1.52137900+2)^{1/3} = 1.52137966. \\ x_5 &= (1.52137966+2)^{1/3} = 1.52137971. \end{aligned}$$

por lo tanto $r \approx 1.52137971$. Solución con R

```
# g(x) = (x + 2)^(1/3)    (x0 = 1)
x <- 1.0
```

```
# Iteración 1
x1 <- (x + 2)^(1/3); x <- x1
```

```
# Iteración 2
x1 <- (x + 2)^(1/3); x <- x1
```

```
# Iteración 3
x1 <- (x + 2)^(1/3); x <- x1
```

```

# Iteración 4
x1 <- (x + 2)^(1/3); x <- x1

# Iteración 5
x1 <- (x + 2)^(1/3); x <- x1

root_aprox <- x
print(root_aprox)

```

Ejemplo 21. $f(x) = \cos x - x = 0$ Se selecciona $x = \cos x$:

$$g(x) = \cos x.$$

$|g'(r)| = |-\sin r| \approx 0.673 < 1$. Tomamos $x_0 = 0.5$.

$$\begin{aligned}
 x_1 &= \cos(0.5) = 0.87758256. \\
 x_2 &= \cos(0.87758256) = 0.63901249. \\
 x_3 &= \cos(0.63901249) = 0.80268510. \\
 x_4 &= \cos(0.80268510) = 0.69477803. \\
 x_5 &= \cos(0.69477803) = 0.76819583.
 \end{aligned}$$

Solución con R

```

# g(x) = cos(x)    (x0 = 0.5)
x <- 0.5

```

```

# Iteración 1
x1 <- cos(x); x <- x1

```

```

# Iteración 2
x1 <- cos(x); x <- x1

```

```

# Iteración 3
x1 <- cos(x); x <- x1

```

```

# Iteración 4
x1 <- cos(x); x <- x1

```

```

# Iteración 5
x1 <- cos(x); x <- x1

```

```

root_aprox <- x
print(root_aprox)

```

Ejemplo 22. $f(x) = e^{-x} - x = 0$, se toma

$$g(x) = e^{-x}.$$

En la raíz $r \approx 0.567143$, $|g'(r)| = e^{-r} \approx 0.567 < 1$. Tomamos $x_0 = 1$.

$$\begin{aligned}
x_1 &= e^{-1} = 0.36787944. \\
x_2 &= e^{-0.36787944} = 0.69220063. \\
x_3 &= e^{-0.69220063} = 0.50047350. \\
x_4 &= e^{-0.50047350} = 0.60624354. \\
x_5 &= e^{-0.60624354} = 0.54539579.
\end{aligned}$$

Solución con R

```
# g(x) = exp(-x)    (x0 = 1)
x <- 1.0
```

```
# Iteración 1
x1 <- exp(-x); x <- x1
```

```
# Iteración 2
x1 <- exp(-x); x <- x1
```

```
# Iteración 3
x1 <- exp(-x); x <- x1
```

```
# Iteración 4
x1 <- exp(-x); x <- x1
```

```
# Iteración 5
x1 <- exp(-x); x <- x1
```

```
root_aprox <- x
print(root_aprox)
```

Algoritmo 11. *Pseudocódigo*

Inicio

```
  Definir f(x) y g(x)
  Leer x0, tolerancia, iter_max
  iter ← 0
  error ← 1
```

```
  Mientras (error > tolerancia) Y (iter < iter_max) Hacer
    x1 ← g(x0)
    error ← |x1 - x0|
    x0 ← x1
    iter ← iter + 1
```

FinMientras

```
  Imprimir "Raíz aproximada:", x1
  Imprimir "Iteraciones:", iter
```

Fin

Implementación numérica:

```
# Método del Punto Fijo
fijo <- function(g, x0, tol = 1e-6, iter_max = 100){
  iter <- 0
  error <- 1

  while(error > tol && iter < iter_max){
    x1 <- g(x0)
    error <- abs(x1 - x0)
    x0 <- x1
    iter <- iter + 1
  }

  cat("Raíz aproximada:", x1, "\n")
  cat("Iteraciones:", iter, "\n")
  return(x1)
}

# Ejemplo de uso:
# f(x) = cos(x) - x → g(x) = cos(x)
g <- function(x) cos(x)
fijo(g, x0 = 0.5)
```

1.5. Ejercicios

1.5.1. Método de Bisección

Instrucciones:

1. Para cada función $f(x)$, encuentre un intervalo $[a, b]$ tal que $f(a) \cdot f(b) < 0$.
2. Realice al menos 8 iteraciones del método de bisección.
3. Registre los valores $a_k, b_k, c_k, f(a_k), f(b_k), f(c_k), \text{signo}(f(a_k)f(c_k))$ y el error $|b_k - a_k|$.
4. Grafique la función y los intervalos de reducción en cada paso.

Ejercicio 1. *Resuelve los siguientes ejercicios aplicando el método de la bisección*

- | | |
|--|---|
| 1. $f(x) = x^2 - 5$ en $[2, 3]$ | 5. $f(x) = \cos(x) - x^2$ en $[0, 1]$ |
| 2. $f(x) = x^3 - 4x + 1$ en $[0, 1]$ | 6. $f(x) = 3x^2 - 7x - 2$. $[a, b] = [-1, 0]$. |
| 3. $f(x) = x^4 - 8x + 2$ en $[0, 3]$ | 7. $f(x) = -4x^2 + 5x + 6$. $[a, b] = [-1, 0]$. |
| 4. $f(x) = e^{-x} - \frac{x}{2}$ en $[0, 2]$ | 8. $f(x) = 2x^2 + 3x - 5$. $[a, b] = [0, 2]$. |

9. $f(x) = x^3 - 5x + 1$. $[a, b] = [0, 1]$. 14. $f(x) = 5e^{0.6x-1} - 3$. $[a, b] = [0, 2]$.
10. $f(x) = x^4 - 6x^3 + 5x^2 + 8x - 4$. $[a, b] = [0, 1]$. 15. $f(x) = 7 - 4e^{-0.9x+0.2}$. $[a, b] = [-2, 0]$.
11. $f(x) = -2x^4 + 4x^3 + 6x^2 - 3x + 2$. $[a, b] = [-2, -1]$. 16. $f(x) = \sin(2x) - \frac{x}{2}$. $[a, b] = [1, 2]$.
12. $f(x) = 2x^4 - x^3 - 7x^2 + 4x + 3$. $[a, b] = [-1, 0]$. 17. $f(x) = (2x + 1.4) \cos\left(3x + \frac{\pi}{4}\right)$. $[a, b] = [0.1, 0.6]$.
13. $f(x) = x^4 + 2x^3 - 10x^2 - x + 5$. $[a, b] = [0, 1]$. 18. $f(x) = (-1.5x + 0.9) \sin\left(4x - \frac{\pi}{6}\right)$. $[a, b] = [0.2, 0.9]$.

Formato sugerido de tabla:

k	a	b	c	$f(a)$	$f(b)$	$f(c)$	$\text{signo}(f(a) * f(c))$	$error$
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
...								

Cuadro 6: Iteraciones del método de bisección para $f(x)$.

1.5.2. Método de la Secante

Instrucciones:

- Para cada función $f(x)$, elija dos puntos iniciales x_0, x_1 cercanos a la raíz.
- Aplique iterativamente:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

- Realice al menos 6 iteraciones o hasta alcanzar una tolerancia de 10^{-5} .
- Compare el número de iteraciones con el método de bisección.
- Grafique las secantes de cada iteración y el punto de intersección con el eje x .

Ejercicio 2. Resuelva los siguientes ejercicios aplicando el método de la secante:

- $f(x) = x^2 - 2x - 3$, con $x_0 = 0$, $x_1 = 3$
- $f(x) = 2x^2 - 5 * x + 3$, $x_0 = -1$, $x_1 = 2$
- $f(x) = x^4 - 10x^2 + 9$, $x_0 = 0$, $x_1 = 2$
- $f(x) = -3x^2 + 4 * x + 2$, $x_0 = -1$, $x_1 = 3$

5. $f(x) = x^3 + 4x^2 - x - 1$, con $x_0 = -3$, $x_1 = 11$ 9. $f(x) = -3e^{-1.2x+0.4} + 2$, con $x_0 = 0$, $x_1 = 1.1$
6. $f(x) = x^4 - 5x^3 + 6x^2 + 4x - 8$, con $x_0 = 0$, $x_1 = 3$ 11. $f(x) = \sin(x) - \frac{x}{3}$, con $x_0 = 0$, $x_1 = 1$
7. $f(x) = x^4 - 5$, con $x_0 = 1$, $x_1 = 2$ 12. $f(x) = (3x + 3.1) \cos(2x + \pi/6)$, con $x_0 = 0.3$, $x_1 = 0.8$
8. $f(x) = -2x^4 + 3x^3 + 7x^2 - 5x + 1$, con $x_0 = 2$, $x_1 = 4$ 13. $f(x) = (-2x + 1.1) \sin(5x - \pi/3)$, con $x_0 = 0.05$, $x_1 = 0.35$
9. $f(x) = 4e^{-0.8x-0.5} - 7$, con $x_0 = 0$, $x_1 = 1.5$

Formato sugerido de tabla:

k	x_{k-1}	x_k	$f(x_k)$	x_{k+1}	$e_k = \ x_{k+1} - x_k\ $
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
...					

Cuadro 7: Iteraciones del método de la secante para $f(x)$.

1.5.3. Método de Newton

Instrucciones: Resuelve los siguientes ejercicios aplicando el **método de Newton**. Para cada función $f(x)$:

- Verifica que exista un intervalo $[a, b]$ donde $f(a) \cdot f(b) < 0$.
- Realiza al menos 8 iteraciones del método.
- Registra los valores $x_k, f(x_k), f'(x_k)$ y el error $|x_{k+1} - x_k|$.
- Grafica la función y marca el punto donde converge la raíz.

- Ejercicio 3.**
- | | |
|---|---|
| 1. $f(x) = x^3 - 7x + 6$, $[a, b] = [0, 1]$ | 7. $f(x) = x^5 - 3x + 1$, $[a, b] = [0, 1]$ |
| 2. $f(x) = x^3 + 2x^2 - 5$, $[a, b] = [1, 2]$ | 8. $f(x) = \cos(x) - 2x$, $[a, b] = [0, 1]$ |
| 3. $f(x) = x^4 - 3x^3 + 2$, $[a, b] = [0, 1]$ | 9. $f(x) = \ln(x + 2) + x - 1$, $[a, b] = [-1, 0]$ |
| 4. $f(x) = \sin(x) - \frac{x}{3}$, $[a, b] = [0, 2]$ | 10. $f(x) = x e^{-x} - 0.1$, $[a, b] = [0, 1]$ |
| 5. $f(x) = e^{-x} - x$, $[a, b] = [0, 1]$ | 11. $f(x) = x^3 - 4 \sin(x)$, $[a, b] = [1, 2]$ |
| 6. $f(x) = x^3 - 2x - 5$, $[a, b] = [2, 3]$ | 12. $f(x) = e^x - 3x^2$, $[a, b] = [0, 1]$ |
| | 13. $f(x) = x^2 - \cos(x)$, $[a, b] = [0, 1]$ |

$$14. f(x) = x^3 + 4x^2 - 10, \quad [a, b] = [1, 2]$$

$$15. f(x) = 2x \sin(x) - 1, \quad [a, b] = [0, 1]$$

Formato sugerido de tabla para Newton:

k	x_k	$f(x_k)$	$f'(x_k)$	$ x_{k+1} - x_k $
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Cuadro 8: Iteraciones del método de Newton para $f(x)$.

1.5.4. Método del Punto Fijo

Instrucciones: Resuelve los siguientes ejercicios aplicando el **método del punto fijo**.

1. Reescriba la ecuación $f(x) = 0$ en forma $x = g(x)$.
2. Verifique que la función $g(x)$ cumpla la condición de convergencia local $|g'(x)| < 1$ en el intervalo considerado.
3. Realice al menos 8 iteraciones partiendo de un valor inicial x_0 .
4. Registre los valores $x_k, g(x_k), |x_{k+1} - x_k|$.
5. Grafique $y = g(x)$ y $y = x$ para visualizar el punto de intersección.

Ejercicio 4. 1. $f(x) = \cos(x) - x$, *Sugerencia:* $g(x) = \cos(x)$, $x_0 = 0.5$

2. $f(x) = x^3 + x - 1$, *Sugerencia:* $g(x) = 1 - x^3$, $x_0 = 0.6$

3. $f(x) = e^{-x} - x$, *Sugerencia:* $g(x) = e^{-x}$, $x_0 = 0.7$

4. $f(x) = x^2 - 4x + 1$, *Sugerencia:* $g(x) = \frac{x^2 + 1}{4}$, $x_0 = 1$

5. $f(x) = x^3 - 2x - 5$, *Sugerencia:* $g(x) = \sqrt[3]{2x + 5}$, $x_0 = 2$

6. $f(x) = x^2 - e^x + 2$, *Sugerencia:* $g(x) = \sqrt{e^x - 2}$, $x_0 = 0.5$

7. $f(x) = \ln(x + 1) + x - 2$, *Sugerencia:* $g(x) = 2 - \ln(x + 1)$, $x_0 = 0.5$

8. $f(x) = x^3 - 3x + 1$, *Sugerencia:* $g(x) = \sqrt[3]{3x - 1}$, $x_0 = 0.8$

9. $f(x) = x - \sin(x) - 1$, *Sugerencia:* $g(x) = \sin(x) + 1$, $x_0 = 1$

10. $f(x) = x^2 - 3$, *Sugerencia:* $g(x) = \frac{3}{x}$, $x_0 = 1.5$

11. $f(x) = 2x - e^{-x}$, Sugerencia: $g(x) = \frac{e^{-x}}{2}$, $x_0 = 0$

12. $f(x) = x^3 + 4x^2 - 10$, Sugerencia: $g(x) = \sqrt{\frac{10 - x^3}{4}}$, $x_0 = 1.5$

13. $f(x) = x^2 - \cos(x)$, Sugerencia: $g(x) = \sqrt{\cos(x)}$, $x_0 = 0.5$

14. $f(x) = x - e^{-x^2}$, Sugerencia: $g(x) = e^{-x^2}$, $x_0 = 0.5$

15. $f(x) = x^3 - 2$, Sugerencia: $g(x) = \sqrt[3]{2}$, $x_0 = 1$

Formato sugerido de tabla para el método del punto fijo:

k	x_k	$g(x_k)$	$ x_{k+1} - x_k $
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Cuadro 9: Iteraciones del método del punto fijo para $f(x) = 0$.

1.5.5. Para impresión

1.5.6. Punto fijo

Pseudocódigo

Algoritmo: Método del Punto Fijo

Entrada:

$g(x)$ → función de iteración
 x_0 → valor inicial
 tol → tolerancia (por ejemplo, $1e-6$)
 $iter_max$ → número máximo de iteraciones

Proceso:

$iter \leftarrow 0$
 $error \leftarrow 1$

Mientras ($error > tol$) Y ($iter < iter_max$) Hacer
 $x_1 \leftarrow g(x_0)$
 $error \leftarrow |x_1 - x_0|$
 $x_0 \leftarrow x_1$
 $iter \leftarrow iter + 1$
FinMientras

Salida:

Imprimir "Raíz aproximada:", x_1
Imprimir "Iteraciones:", $iter$

FinAlgoritmo

Implementación numérica

```
\begin{lstlisting}[language=R, caption={Método del Punto Fijo en R}]
```

```
fijo <- function(g, x0, tol = 1e-6, iter_max = 100){
```

```
  iter <- 0  
  error <- 1
```

```
  while(error > tol && iter < iter_max){  
    x1 <- g(x0)  
    error <- abs(x1 - x0)  
    x0 <- x1  
    iter <- iter + 1  
  }
```

```
  cat("Raíz aproximada:", x1, "\n")  
  cat("Iteraciones:", iter, "\n")  
  return(x1)
```

```
}
```

```
# Ejemplo de uso:
```

```

g <- function(x) cos(x)
fijo(g, x0 = 0.5)
\end{lstlisting}

```

1.5.7. Newton-Raphson

Pseudocódigo

Algoritmo: Método de Newton-Raphson

Entrada:

```

f(x)      → función
f'(x)     → derivada de f(x)
x0        → valor inicial
tol       → tolerancia
iter_max  → número máximo de iteraciones

```

Proceso:

```

iter ← 0
error ← 1

Mientras (error > tol) Y (iter < iter_max) Hacer
    x1 ← x0 - f(x0)/f'(x0)
    error ← |x1 - x0|
    x0 ← x1
    iter ← iter + 1
FinMientras

```

Salida:

```

Imprimir "Raíz aproximada:", x1
Imprimir "Iteraciones:", iter

```

FinAlgoritmo

Implementación numérica

```

newton <- function(f, df, x0, tol = 1e-6, iter_max = 100){
  iter <- 0
  error <- 1

  while(error > tol && iter < iter_max){
    x1 <- x0 - f(x0)/df(x0)
    error <- abs(x1 - x0)
    x0 <- x1
    iter <- iter + 1
  }
}

```

```

cat("Raíz aproximada:", x1, "\n")
cat("Iteraciones:", iter, "\n")
return(x1)
}

```

Ejemplo:

```

f <- function(x) x^2 - 2
df <- function(x) 2*x
newton(f, df, x0 = 1)

```

1.5.8. Bisección

Pseudocódigo

Algoritmo: Método de Bisección

Entrada:

```

f(x)      → función continua
a, b      → intervalo [a,b] con f(a)*f(b) < 0
tol       → tolerancia
iter_max  → número máximo de iteraciones

```

Proceso:

```

Si f(a)*f(b) > 0 Entonces
    Imprimir "No hay cambio de signo"
    Salir
FinSi

```

```

iter ← 0
error ← |b - a|

```

```

Mientras (error > tol) Y (iter < iter_max) Hacer
    c ← (a + b)/2

```

```

    Si f(a)*f(c) < 0 Entonces
        b ← c
    Sino
        a ← c
    FinSi

```

```

    error ← |b - a|
    iter ← iter + 1
FinMientras

```

Salida:

```

Imprimir "Raíz aproximada:", c

```

```
    Imprimir "Iteraciones:", iter
FinAlgoritmo
```

Implementación numérica

```
biseccion <- function(f, a, b, tol = 1e-6, iter_max = 100){
  if(f(a)*f(b) > 0){
    cat("Error: no hay cambio de signo en [a,b].\n")
    return(NA)
  }

  iter <- 0
  error <- abs(b - a)

  while(error > tol && iter < iter_max){
    c <- (a + b)/2
    if(f(a)*f(c) < 0){
      b <- c
    } else {
      a <- c
    }
    error <- abs(b - a)
    iter <- iter + 1
  }

  cat("Raíz aproximada:", c, "\n")
  cat("Iteraciones:", iter, "\n")
  return(c)
}
```

Ejemplo:

```
f <- function(x) x^3 - x - 2
biseccion(f, a = 1, b = 2)
```

1.5.9. Secante

Pseudocódigo

Algoritmo: Método de la Secante

Entrada:

$f(x)$	→ función
x_0, x_1	→ valores iniciales
tol	→ tolerancia
$iter_max$	→ número máximo de iteraciones

Proceso:


```

iter ← 0
error ← 1

Mientras (error > tol) Y (iter < iter_max) Hacer
    fx0 ← f(x0)
    fx1 ← f(x1)

    Si |fx1 - fx0| < 1e-12 Entonces
        Imprimir "Error: división por cero"
        Salir
    FinSi

    x2 ← x1 - fx1 * (x1 - x0) / (fx1 - fx0)
    error ← |x2 - x1|

    x0 ← x1
    x1 ← x2
    iter ← iter + 1
FinMientras

Salida:
    Imprimir "Raíz aproximada:", x2
    Imprimir "Iteraciones:", iter
FinAlgoritmo

Implementación numérica

secante <- function(f, x0, x1, tol = 1e-6, iter_max = 100){
    iter <- 0
    error <- 1

    while(error > tol && iter < iter_max){
        fx0 <- f(x0)
        fx1 <- f(x1)

        if(abs(fx1 - fx0) < 1e-12){
            cat("Error: División por cero (f(x1) - f(x0) approx 0)\n")
            return(NA)
        }

        x2 <- x1 - fx1 * (x1 - x0) / (fx1 - fx0)
        error <- abs(x2 - x1)

        x0 <- x1
        x1 <- x2
        iter <- iter + 1
    }

    cat("Raíz aproximada:", x2, "\n")

```

```
    cat("Iteraciones:", iter, "\n")
    return(x2)
}

# Ejemplo:
f <- function(x) x^3 - 2*x^2 + 3*x - 5
secante(f, x0 = 1, x1 = 2)
```

1.6. Tarea para el portafolio

A continuación se presentan las indicaciones sobre los ejercicios que se van a incluir en el portafolio del curso de Métodos Numéricos

1. Del método de Bisección, (subsección 1.5.1) del ejercicio 1, resolver de los numerales impares: 3 ejercicios *a mano* y 5 ejercicios con el *código automatizado*.
2. Del método de la Secante (subsección 1.5.2), del ejercicio 2, resolver de los ejercicios impares 3: *a mano* y de los ejercicios pares 5: con el *código automatizado*.
3. De la subsección 1.5.3, Del método de Newton (subsección 1.5.3), del ejercicio 3, resolver de los ejercicios pares: 3 *a mano* y de los ejercicios impares 5: con el *código automatizado*.
4. De la subsección 1.5.4, Del método del Punto Fijo (subsección 1.5.4), del ejercicio 4, resolver de los: 3 *a mano* y 5 con el *código automatizado*.